Call, Agent, or Team? Orchestration in Practice

ME/CE/AM 295

Agentic AI in Scientific Discovery
9 units (3-5-1)

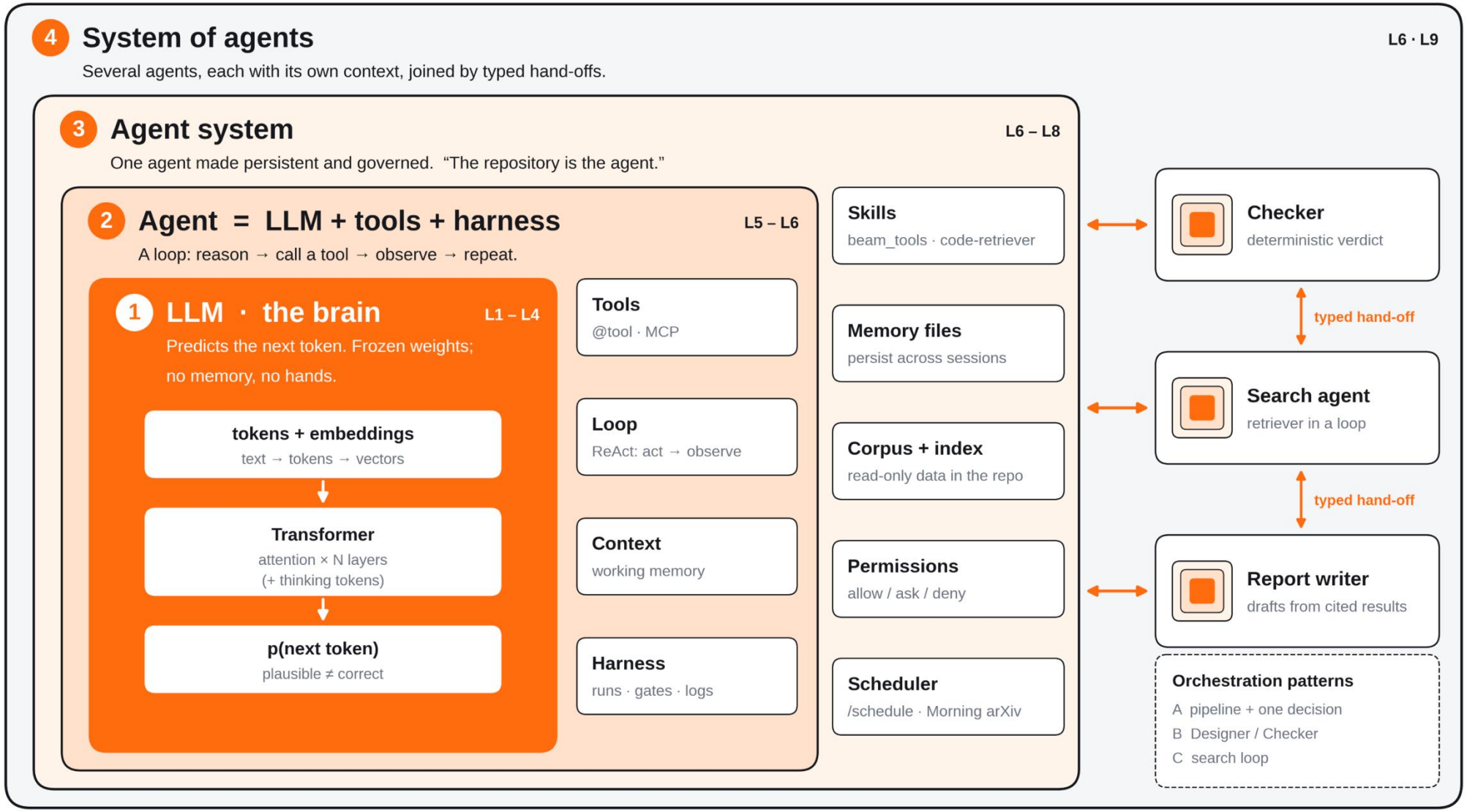
Instructor: Chiara Daraio
TAs: Xiaoxiao (Alice) Xiong
Hyoseo (Lauren) Yoon

California Institute of Technology
Division of Engineering and Applied Science

Lecture 10

Monday 2 November 2026

The Class, So Far



WHERE WE LEFT OFF · WEDNESDAY (LECTURE 9)

We stopped at MechAgents: a critic caught a force of 0.0 that a lone agent accepted.
Today: five decisions of your own, the rest of the toolkit, and where to build your own team.



PART 1	PART 2	PART 3	PART 4	IF TIME
Call, agent, or team?	The evidence	Design the structure	Tools: where to build	In the field

0:00 recap → 0:02 five scenarios → 0:32 evidence → 0:45 structure → 0:57 tools → 1:06 close → if time: in the field

Call, Agent, or Team? — Climb Only as High as You Must

Five rungs. Each one up buys flexibility and costs tokens, latency, and a new way to fail.

0 SCRIPT

No LLM: a formula, a parser, a solver.

tokens vs a chat: 1×

1 LLM CALL

One judgment on text, schema out (Lecture 3). Your code loops over the files.

tokens vs a chat: 1×

2 WORKFLOW

Several calls; the path is fixed in your code: chain, route, parallel (Lecture 9 patterns 2–6).

tokens vs a chat: few ×

3 ONE AGENT

The model picks the next step: tools, a loop, a stop (Lecture 5).

tokens vs a chat: ≈ 4×

4 A TEAM

Two or more contexts, each earned by a reason (Lecture 9, Rule 3).

tokens vs a chat: ≈ 15×

Q1 **Is there a D step — a point where you read an output and decide?** No → rung 0 or 1.

Q2 **Can you write the path in advance?** Yes → rung 1–2.

Q3 **Does a second context earn its place — INFORMATION, PERMISSIONS, JUDGMENT?** No → rung 3.

Cost multipliers: Hadfield et al. 2025 (one agent ≈ 4× a chat, a multi-agent system ≈ 15×). Rungs 1–2 follow Building Effective Agents (Schluntz & Zhang 2024). Vote on each scenario by rung: 0–4.

Scenario A — The Tensile-Report Table

THE SITUATION

Your lab has 480 PDF tensile-test reports (2014–2026) from one test machine, all on the vendor's template. You want one table: specimen ID, E, 0.2 % offset yield, UTS, elongation. Each report also has a free-text "operator remarks" box ("slipped in grip", "broke outside gauge", "OK") that you want sorted into categories.

FACTS THAT MATTER

- Fixed template: every number sits in the same field of every PDF.
- Remarks are short, messy, sometimes blank.
- Raw load–displacement data exist for about half the reports.
- A wrong yield value would end up in your thesis table.

VOTE **0 script · 1 LLM call · 2 workflow · 3 one agent · 4 a team**

ASK What decides the rung — the 480 files, or the remarks box?

Scenario B — The Diverging Pushover

THE SITUATION

An OpenSeesPy nonlinear pushover of a four-storey steel moment frame stops converging at about 2 % roof drift.

Candidate causes: the integrator, the step size, the convergence test and tolerance, the element formulation, P-Δ, the steel material model. Each run takes three minutes and writes a log.

FACTS THAT MATTER

- Each fix you try depends on what the last log said.
- The tools exist: run_model, read_log, set_param.
- The target drift (4 %) comes from the design spec.
- One easy "fix" is to loosen the tolerance until it converges.

VOTE **0 script · 1 LLM call · 2 workflow · 3 one agent · 4 a team**

ASK Someone proposes a Diagnoser agent, a Fixer agent and a Reviewer agent. Better or worse than one?

Scenario C — The Architected-Materials Survey

THE SITUATION

For a review paper you want a map of published approaches to impact-energy absorption in architected materials, 2015–2026, across six families: strut lattices, TPMS, origami, multistable, granular, hierarchical. About 400 papers.

Output: one table — mechanism, base material, strain rate, specific energy absorption, citation.

FACTS THAT MATTER

- The six families can be read independently of each other.
- 400 papers will not fit in one context window, even as abstracts plus results.
- You built a retriever and check_citations in Lab 5.
- Some papers belong to two families.

VOTE **0 script · 1 LLM call · 2 workflow · 3 one agent · 4 a team**

ASK Is this the first scenario that earns a second agent? Which reason — and what must the brief say?

Scenario D — The Fatigue-Data Pipeline

THE SITUATION

You are building an S-N fatigue database from about 300 papers. Some report experiments, some report simulations, some are reviews, and each type needs different fields: experiments need material, R-ratio, stress amplitude, cycles to failure and specimen geometry; simulations need model, mesh, boundary conditions and predicted life. Every row must be checked before it enters the database.

FACTS THAT MATTER

- The paper types are known in advance: experimental, simulation, review, irrelevant.
- Each type needs its own extraction schema.
- A script can check each row: $\sigma_{max} \leq$ UTS; R matches the reported σ_{min} and σ_{max}; within one series, log N falls as stress rises.
- You can draw the whole flowchart before you run anything.

VOTE **0 script · 1 LLM call · 2 workflow · 3 one agent · 4 a team**

ASK One call, a fixed chain of calls, or an agent that decides what to read next?

S-N curve: stress S (usually the amplitude σ_a) vs the number of cycles to failure N.
R = σ_{min} / σ_{max}: the stress ratio (R = −1 fully reversed; R = 0.1 tension–tension).

Scenario E — The Second Pair of Eyes

THE SITUATION

Before you submit a paper, you want an LLM to review your finite-element model of a bolted joint: mesh, boundary conditions, contact definitions, material cards. You built the model with Claude Code over three weeks; that session remembers every choice you made and why.

FACTS THAT MATTER

- The building session has read all your reasoning.
- A reviewer that agrees with you is worth little; one that finds a real error is worth a lot.
- Both reviewers would run the same model.
- Mesh convergence and reaction-force equilibrium can be checked by script.

VOTE **0 script · 1 LLM call · 2 workflow · 3 one agent · 4 a team**

ASK Ask the building session to review its own work, or start a reviewer that has never seen the project?

Five Scenarios, Five Answers — the Reason, Then the Check

The rung follows from the decision structure, not from the size or difficulty of the task.

Scenario	Rung	Why this rung — and not one higher	The check that is not the LLM
A · Tensile reports	0 + 1	Fixed template → parser; remarks → one call with a schema. No D step. Volume is not breadth.	Yield recomputed from raw curves; 30 hand-labelled remarks
B · Diverging pushover	3	Next try depends on the last log; sequential, one shared context. A team would lose context at every hand-off.	Converged to 4 % drift + base shear vs hand estimate; tolerance fixed in the harness
C · Materials survey	4	BREADTH: six independent families, more than one context holds. Orchestrator + readers + one writer.	check_citations at each hand-off; same-budget single-agent baseline
D · Fatigue-data pipeline	2	Path known in advance: route by type → typed extraction → script gate → one retry. Code decides every arrow.	Range, unit and S-N monotonicity validator; 30 hand-labelled papers
E · Second pair of eyes	3 + reviewer	JUDGMENT: a fresh context that has not read your reasoning — but the same model, so it may flag, never pass.	Mesh convergence, reaction-force equilibrium, energy balance, a colleague

A second agent buys a separate context — nothing else. Name the reason, or stay one rung lower.

Rung 3 + reviewer = one agent plus a second context that is a read-only subagent (E). Together the five answers cover every rung of the ladder.

Multi-Agent Science — Read the Check Column First

Four teams, from the strongest check to the weakest. Where the check is a proof, an experiment or a script, the claim is strong; where it is another LLM, you have bought a second faithfulness problem.

System	The team	What checks the output
Forced Navier–Stokes blowup **OpenAI, Sep 2026**	~10,000 agents in groups that message each other, each attacking a variant of the problem	A Lean formal proof — but humans must confirm the formal statement is the Clay problem
Virtual Lab **Swanson et al. 2025, Nature**	PI agent + specialist agents + a scientific critic; a human sets the agenda	Wet lab: designed nanobodies expressed and tested
MechAgents **Ni & Buehler 2024 (Lecture 9)**	Planner, coder, executor and critic in one group chat, on FE elasticity problems	An LLM critic, plus a human hint; case studies, no repeated runs
Design Agents **Elrefaie et al. 2025**	Styling, CAD and aerodynamics agents for car design	A drag surrogate trained on DrivAerNet++ — a model, not CFD

Each row is also a design choice: who orchestrates and what the team shares. Navier–Stokes figures as reported by OpenAI (via Quanta, 8 Sep 2026); the result is under review and the Clay problem is still listed as open.

The Evidence — Teams Buy Tokens, Not Intelligence

The strongest numbers, from those building the systems and from controlled comparisons.

+90%

Lead + parallel subagents vs one agent, on an internal research eval of breadth-first questions.

Hadfield et al. 2025

~15×

the tokens of a chat (one agent ≈ 4×). Token use alone explained 80% of the variance on BrowseComp.

Hadfield et al. 2025

+81% … −70%

From decomposable financial reasoning to sequential planning — same tools, prompts, compute.

Kim et al. 2025 · 260 configs

14 modes

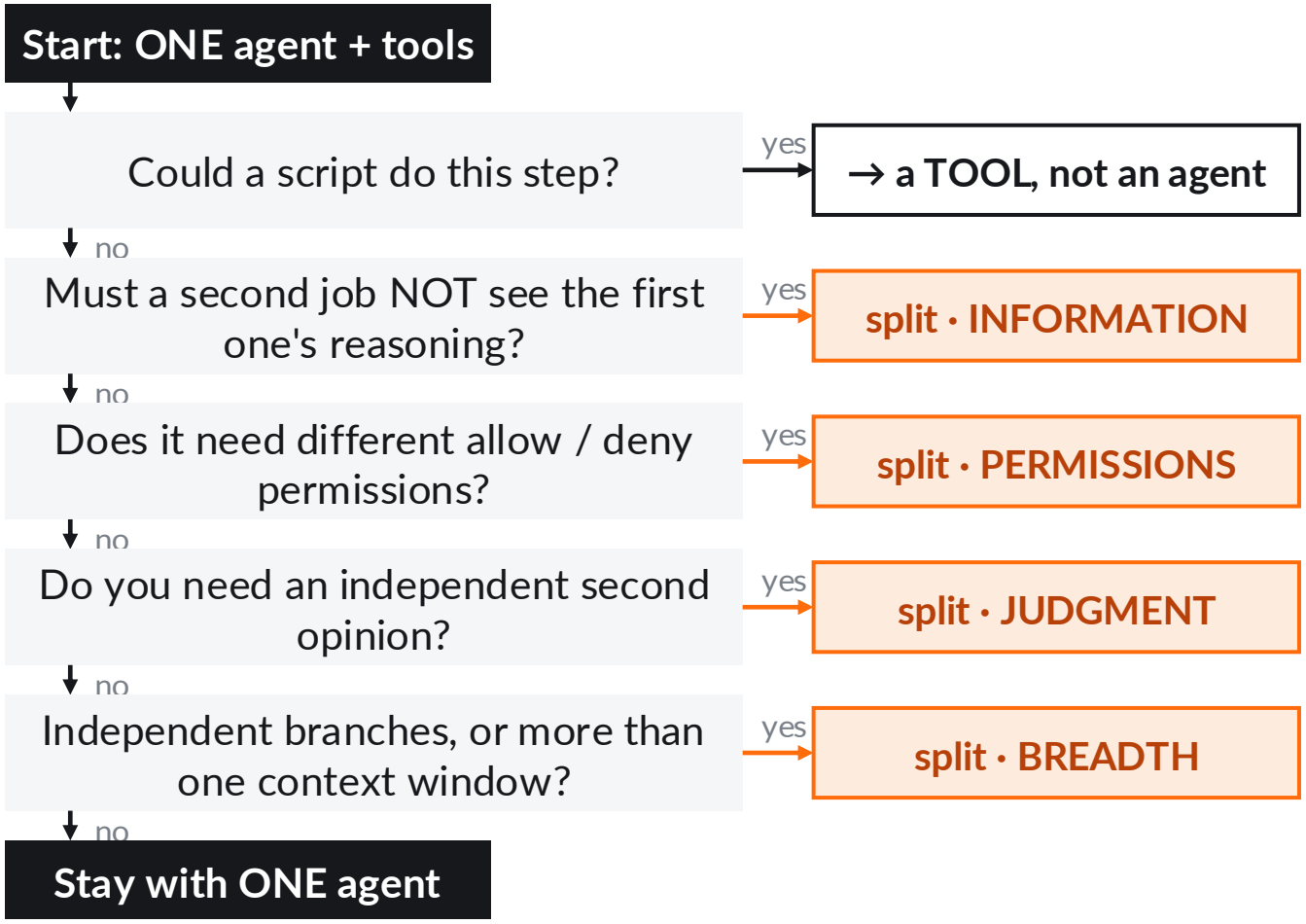
in 1,600+ traces; most failures are design and inter-agent misalignment, not the model.

Cemri et al. 2025 · MAST

DISCUSSION POINT 1 If tokens explain 80% of the variance, what single-agent baseline must you run before you can claim your team helps?

Also: multi-agent debate does not reliably beat self-consistency at equal budget (Smit et al. 2024).
Anthropic figure: Claude Opus 4 lead with Sonnet 4 subagents vs Opus 4 alone.

One Agent or Several? — The Split Test



DO NOT SPLIT FOR

Sequential chains where each step needs the last one's full context: teams lose up to 70% here.

Pipeline stages (load, run, tabulate): those are tools.

Personas ("you are a materials scientist"): a prompt, not an agent.

SPLIT THE READING, NOT THE WRITING

Read in parallel: one reader per branch (Scenario C).

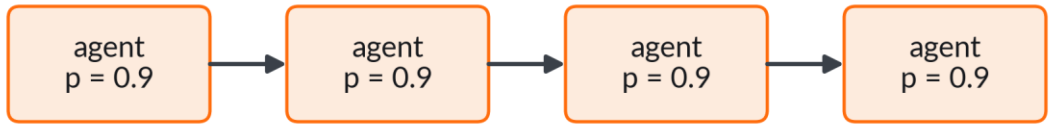
Write in one place: one agent owns it (Scenario B).

BREADTH = independent branches, or more material than one context window holds.

Why Teams Fail Quietly — Series Systems and the Inspector

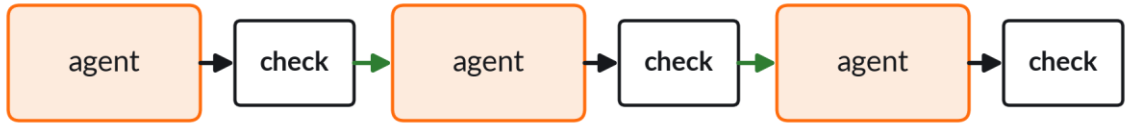
A chain of agents is a series system: it is as reliable as the product of its links. The fix structural engineers use is the one that works here — inspect at the joints.

SERIES — every hand-off unchecked



$0.9^4 \approx 0.66$ — the rule of three, extended

CENTRALIZED — a check at each hand-off



Independent agents: errors ×17.2

Centralized, with validation: ×4.4

Kim et al. 2025 (arXiv v1)

THE COST LINE GROWS TOO

Lecture 4: Cost = N_calls · N_samples ×[p_in·n_in + p_out·(n_answer + n_reasoning)]. Every agent adds calls and re-reads context: ≈ 15 ×a chat.

Beam example: Designer → Reviewer → Reporter, three LLMs, no script: $0.9^3 \approx 0.73$. Designer → script Checker: the joint is inspected.

DISCUSSION POINT 2 Worksheet box 6: which of the four reasons does your second agent meet? If none, cross it out.

×17.2 vs ×4.4 error amplification: Kim et al. 2025, arXiv v1; v3 (Apr 2026) restates it qualitatively — architectures without centralized verification propagate more errors.

In the News This Month — Opposite Moves, Same Week

October 2026: Meta split one agent into fourteen; Anthropic advised folding subagents back into one loop. Both are right, for the same reason.

ONE → FOURTEEN · META 19 Oct

- One agent that knew everything hallucinated in production: right knowledge, applied to the wrong subsystem.
- Fix: a 1,300-line router that does zero domain work, plus 14 specialists, each with isolated context.
- 6 of 7 steps are deterministic code; only the delegated step uses the LLM.
- Reported: column prep 3–4 days → under an hour; about 2/3 of changes land without major edits.

MANY → ONE · ANTHROPIC 22 Oct

- Enterprise agents built as a copy of the org chart: one subagent per team.
- Symptoms: subagent sprawl, a "Frankenprompt", lossy hand-offs, added latency.
- Fix: one loop that loads skills and tools only when needed; one customer ran about 2×faster.
- Keep a subagent for context bloat, parallel work, or tool-selection confusion.

Both split by context, not by org chart. Meta's specialists need knowledge that confuses the others (INFORMATION); Anthropic's shared context and only paid hand-off costs.

Engineering at Meta (Rath et al.), "The Thin Orchestrator" (19 Oct 2026) · Anthropic, "Building agents: When to fold subagents into your main agent loop" + single-agent-loop-kit (22 Oct 2026). Company-reported figures; neither gives a sample size.

Three Questions, Answered on Purpose

For each question, options run from most control in your code (left) to most control in the model (right). The course default is the column you start from.

	Your code ←		→ The model	Course default
Control flow	fixed sequence	graph with conditional edges	orchestrator LLM delegates; peers hand off	graph; an LLM orchestrator only for BREADTH
Shared state	private context + a brief	shared files (a blackboard)	one shared transcript	files in the repo, one writer each; pass paths, not copies
Hand-off	typed schema + script check	typed schema	free text	typed and checked — a stamped submittal

INSTRUCTION HIERARCHY

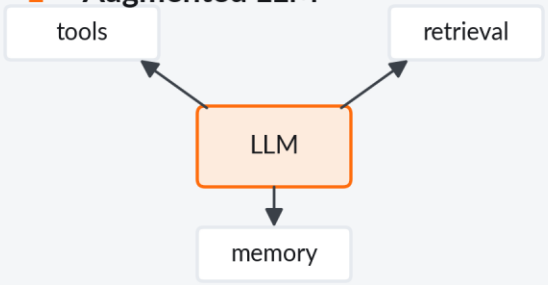
Spec and harness > orchestrator's brief > a worker's output > retrieved text. A worker's output is data, never an instruction (Wallace et al. 2024) — Lecture 7's poisoning lesson, one level up.

Hand-off row is ordered by control: a script-checked schema leaves the model least room. Passing references to files instead of copies: Hadfield et al. 2025, appendix.

Seven Patterns — 1 Building Block, 5 Workflows, 1 Agent

Building Effective Agents (Schluntz & Zhang, Anthropic, Dec 2024) names the shapes. You have built four of them; orchestrator–workers is the new one today.

1 Augmented LLM



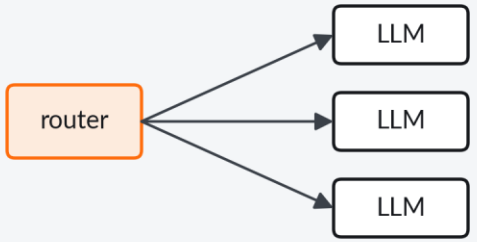
building block · L2–L3

2 Prompt chaining

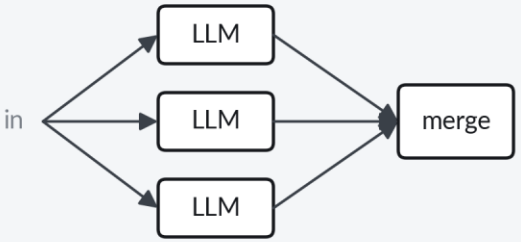


L2 pipeline

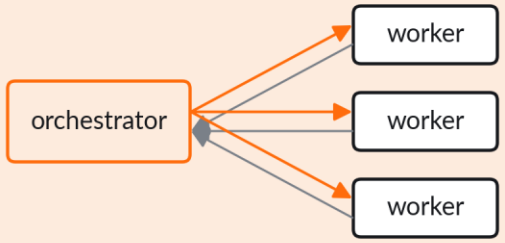
3 Routing



4 Parallelization



5 Orchestrator–workers



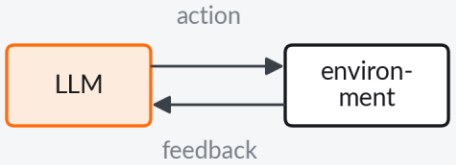
today · new

6 Evaluator–optimizer



L6 Designer/Checker

7 Autonomous agent



L5 loop · L8 search

WORKFLOWS (2–6)

code decides the path

AGENT (7)

the model decides the path

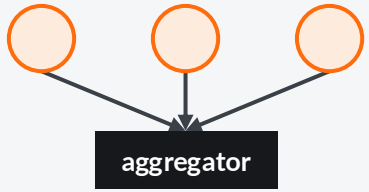
Schluntz & Zhang 2024

Multi-agent topologies map onto these: supervisor = 5 · hierarchical = 5 nested · debate = 4 + 6 · peer hand-off ("swarm", Kim et al.'s decentralized) = 7 with several LLMs.

Four Ways to Organise a Team — Who Talks to Whom

Same tasks, tools and compute (Kim et al. 2025). What differs: who talks to whom, and who checks.

INDEPENDENT

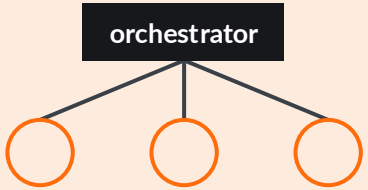


Who talks: Agents report only to an aggregator; outputs are stitched, never cross-checked.

Kim et al.: Errors ×17.2. Worst: −70 % on sequential planning.

In this lecture: Tile 4 (parallelization) with no checker.

CENTRALIZED

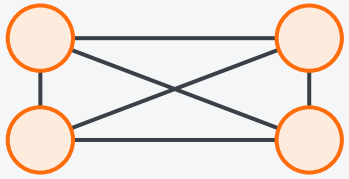


Who talks: One orchestrator delegates, collects and checks.

Kim et al.: Errors ×4.4. Best: +80.8 % on decomposable financial analysis. The hub is a bottleneck.

In this lecture: Tile 5; slide 20; Scenario C.

DECENTRALIZED

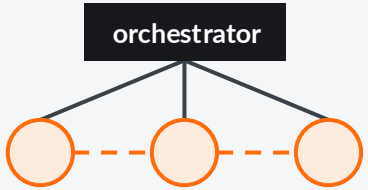


Who talks: Every agent talks to every other, in debate rounds.

Kim et al.: Helped wide exploration: web navigation +9.2 %.

In this lecture: Debate; "swarm"; flat peers.

HYBRID



Who talks: An orchestrator plus limited peer-to-peer messages.

Kim et al.: Hub control plus lateral exchange; more messages.

In this lecture: The Navier–Stokes run (slide 11).

Put a checker at the centre. Match the architecture to the task: all four lost on sequential planning.

Kim, Y. et al. (2025). Towards a science of scaling agent systems. arXiv:2512.08296 v3 (Apr 2026): 260 configurations, same tools, prompts and compute. Communication topology is one design dimension among several.

MAST — Fourteen Ways a Team Fails, in Three Families

MAST = Multi-Agent System Failure Taxonomy. Cemri et al. annotated 1,600+ traces from seven frameworks. The families tell you where to put the fix.

SPECIFICATION / SYSTEM DESIGN · 5

disobeys task or role spec · repeats steps · loses history · doesn't know when to stop

Fix: a written spec per agent; step budget; stop condition in the harness

INTER-AGENT MISALIGNMENT · 6

conversation reset · no clarifying question · derails · withholds · ignores input · reasoning ≠ action

Fix: typed brief with required fields; artifacts in files; "ask" as an allowed output

TASK VERIFICATION · 3

stops too early · no or incomplete verification · incorrect verification

Fix: a deterministic checker outside the team; threshold from the spec

```
[designer] proposes W10x22 for the 20 ft span
[checker]  "W10x22 looks reasonable. PASS."
           (no tool call in the trace)
```

DISCUSSION POINT 3 Which MAST mode is this — and does it matter that W10×22 is in fact the right answer?

Full list of the 14 modes: Appendix A. Mode names follow arXiv:2503.13657 v3 (NeurIPS 2025).

The Orchestration Contract, in Code

An orchestrator–workers loop in the raw SDK. Every rule on the next slide is a line here; none of them is a sentence in a prompt.

```
class Brief(BaseModel):                # the submittal
    objective: str
    output_schema: str                 # e.g. "Finding"
    allowed_tools: list[str]
    boundaries: str                    # what NOT to do
    max_tool_calls: int = 10

class Plan(BaseModel):
    briefs: list[Brief]                # 1-4; more needs a reason

for rnd in range(MAX_ROUNDS):          # MAX_ROUNDS = 3
    plan = client.messages.parse(model=MODEL, max_tokens=4000,
        system=ORCH_SPEC, messages=history,
        output_format=Plan).parsed_output
    found = [run_worker(b) for b in plan.briefs]  # own context
    verdict = check(found)             # a script, not a model
    spent += sum(f.tokens for f in found)
    if verdict.ok or spent > BUDGET:
        break
    history.append(feedback(verdict))
if not verdict.ok:
    escalate("needs_human", found)     # Lecture 6: k = 3
```

`Brief`
the hand-off is a type: objective, schema, tools, boundaries, budget

`run_worker`
a fresh context per brief — the worker never sees the plan's reasoning

`check`
deterministic; the orchestrator cannot talk it round

`MAX_ROUNDS · BUDGET`
stop conditions live in code, not in the prompt

`needs_human`
the honest exit: a person decides

MODEL = the course model ID (pinned in the repo). run_worker() is the Lecture 5 tool loop with its own messages list; Finding is the Lecture 8 Grounded schema.

Five Rules for Orchestration

Each rule removes one family of failures from slide 18 — and each is enforced by the harness, not requested of the model.

1. **Start with one agent and tools.** Add an agent only for INFORMATION, PERMISSIONS, JUDGMENT or BREADTH — and beat a single agent on the same budget.
2. **Code owns control flow wherever you can write it.** The model decides the path only where you cannot.
3. **Every hand-off is a typed brief.** Objective, output schema, tools, boundaries, budget — a submittal, not a chat message.
4. **One writer per artifact.** Agents share files, not transcripts; pass paths, not copies.
5. **Checks and stops live in the harness.** A deterministic checker at each joint; max rounds, a token budget, and needs_human.

A rule the model reads is a suggestion; a rule the harness reads is a rule.

The Tools — Four You Already Have

A framework is a harness you didn't write (Lecture 6). Choose by where you want the control flow to live, and what you need to replay.

Tool	You define the team as	Control flow lives in	Use it when
Claude Code subagents	Markdown files in .claude/agents/ — own prompt, tools, model	the main agent, which delegates; a subagent returns only its result	capstone default: the checker on slide 23
Claude Agent SDK	Python: the Claude Code harness as a library, with subagents	the lead agent, inside your script	your agent becomes a program or a CI job
Raw Anthropic SDK	your own loop (slide 19)	your code	small teams you must audit line by line
LangGraph	StateGraph: nodes, edges, a typed shared State, a checkpointer	an explicit graph; supervisor or hand-off edges	you need replay, resume and human gates (Lecture 6)

Docs accessed October 2026: code.claude.com (subagents; agent teams for parallel sessions that message each other) · Claude Agent SDK · LangGraph multi-agent.

The Tools You Will Meet in Papers — and Two Protocols

Most multi-agent science papers since 2023 are built on one of these. Read them for their control flow, and translate.

AutoGen → Microsoft Agent Framework

Agents converse in a group chat; a manager picks the next speaker. Its successor adds typed graph workflows; AutoGen itself is in maintenance.

control flow: a chat manager

CrewAI

A crew of agents with role, goal and backstory; tasks run in a sequential or hierarchical process; Flows add explicit steps.

control flow: the process type

neuro-san (Cognizant AI Lab)

Agent networks declared in HOCON config files; agents route requests among themselves; private data travels outside the prompt (sly_data).

control flow: the network file

MCP · AGENT ↔ TOOL

Lecture 5. How a model finds and calls a tool: N×M integrations become N+M.

A2A · AGENT ↔ AGENT

How agents built by different vendors discover each other and exchange tasks. Both now sit under the Linux Foundation.

All three are model-agnostic; in this course run them with the pinned course model. Wu et al. 2023 (AutoGen); learn.microsoft.com/agent-framework; github.com/cognizant-ai-lab/neuro-san.

Register It — Your Agent's First Subagent

The Lecture 6 Checker, as a file in your capstone repo. Then the same split as a graph, for comparison.

`.claude/agents/checker.md`

```
---
name: checker
description: Verifies a Designer hand-off.
  Use after every design proposal.
  Never edits files.
tools: Read, Bash
model: inherit
---
You are the Checker. Run beam_tools.py
check on the proposed section. Return
PASS / FAIL / ABSTAIN with the tool output.
No tool output means ABSTAIN.
```

The same split in LangGraph (Lecture 6)

```
g = StateGraph(BeamState)
g.add_node("designer", designer)    # LLM
g.add_node("checker", run_checker)  # script
g.add_edge(START, "designer")
g.add_edge("designer", "checker")
g.add_conditional_edges("checker", route,
    {"revise": "designer",
     "done": END, "needs_human": END})
```

Information split: the checker sees the brief and the design, not the Designer's reasoning. Permission split: Read + the beam_tools command only — enforced in settings.json.

Subagents run in their own context window and return only their result. Optional this week; in the proposal, worksheet box 6 points at this file.

How Do You Know the Team Is Right?

A trace is evidence, not proof. A team leaves several traces — read the hand-offs first.

SAME-BUDGET BASELINE

One agent, same tools, same token budget, same held-out questions. The team must beat it (Discussion Point 1).

CHECKS AT EVERY JOINT

Count PASS / FAIL / ABSTAIN at each hand-off, not only at the end.

ONE TRACE PER AGENT

Tag each line with the agent that wrote it; 20 runs; label failures with MAST codes.

THE needs_human RATE

How often the team stops and asks. Zero is as suspicious as always.

Returns in Lecture 16 (evaluation, reliability, deployment) and the Week 9 audit of your persistent agent.

Lecture 10 — Takeaways, Readings, and Wednesday

Climb the ladder Script → LLM call → workflow → agent → team. Go only as high as the decision structure needs. Volume is not breadth.

One or many Split only for information, permissions, judgment or breadth — and beat one agent on the same budget.

Design Typed briefs, one writer per artifact, checks and stops in the harness.

Where to build Claude Code subagents for the capstone; the Agent SDK when it becomes a program; the raw SDK to audit line by line; LangGraph for replay and human gates.

Discussion Points: 1 same-budget baseline · 2 your box 6 · 3 the checker that never checked · 4 (Appendix B, if time) what would you add to the scorecard

READINGS

Required Hadfield et al. 2025 · Cemri et al. 2025 § § 1–4 · Carlini 2026 (C compiler)

Recommended Mishra-Sharma 2026 · Lin 2026 (Cursor) · Kim et al. 2025 · Yan 2025

Optional Project Vend phase two (2025) · Lin et al. 2026 (kernels) · Smit et al. 2024

WEDNESDAY 4 NOVEMBER · LECTURE 11

TA-led workshop: four project talks (10 + 5 min), then peer review of proposals in small groups.
Mon 9 Nov: physics-informed surrogates. Guide & Check due Sun 8 Nov.

References — Carried from Lecture 9

Smith, R. G. (1980). The Contract Net Protocol. IEEE Trans. Computers C-29(12), 1104–1113.

Erman, L. D., Hayes-Roth, F., Lesser, V. R., Reddy, D. R. (1980). The Hearsay-II speech-understanding system. ACM Computing Surveys 12(2), 213–253.

Minsky, M. (1986). The Society of Mind. Simon & Schuster. · Wooldridge, M., Jennings, N. R. (1995). Intelligent agents: theory and practice. Knowledge Eng. Review 10(2).

Li, G. et al. (2023). CAMEL. NeurIPS. · Wu, Q. et al. (2023). AutoGen. arXiv:2308.08155. · Hong, S. et al. (2024). MetaGPT. ICLR. · Qian, C. et al. (2024). ChatDev. ACL.

Du, Y. et al. (2024). Improving factuality and reasoning through multiagent debate. ICML. · Smit, A. P. et al. (2024). Should we be going MAD? ICML, PMLR 235.

Ni, B., Buehler, M. J. (2024). MechAgents: Large language model multi-agent collaborations can solve mechanics problems, generate new data, and integrate knowledge. Extreme Mechanics Letters 67, 102131.

Schluntz, E., Zhang, B. (2024). Building Effective Agents. Anthropic [Ref_20]. · Hadfield, J. et al. (2025). How we built our multi-agent research system. Anthropic, 13 Jun 2025.

Yan, W. (2025). Don't build multi-agents. Cognition, Jun 2025. · Cemri, M. et al. (2025). Why do multi-agent LLM systems fail? NeurIPS 2025; arXiv:2503.13657 [Ref_25].

Kim, Y. et al. (2025). Towards a science of scaling agent systems. arXiv:2512.08296 (v1 Dec 2025; v3 Apr 2026).

Swanson, K. et al. (2025). The Virtual Lab of AI agents designs new SARS-CoV-2 nanobodies. Nature. · Gottweis, J. et al. (2025). Towards an AI co-scientist. arXiv:2502.18864.

Ghafarollahi, A., Buehler, M. J. (2024). SciAgents. Advanced Materials, doi:10.1002/adma.202413523. · Elrefaie, M. et al. (2025). AI agents in engineering design: a multi-agent framework for aesthetic and aerodynamic car design. ASME IDETC 2025; arXiv:2503.23315 [Ref_05].

Wallace, E. et al. (2024). The instruction hierarchy. arXiv:2404.13208.

In the news: Rath, P., Jindal, A., Verma, A. (2026). The thin orchestrator: how we made our AI agents more reliable by giving the router less to do. Engineering at Meta, 19 Oct. · Anthropic (2026). Building agents: when to fold subagents into your main agent loop, 22 Oct (+ github.com/anthropics/single-agent-loop-kit). · Anthropic (2026). Dynamic Workflows, Claude Managed Agents docs (beta, 9 Oct). · Zhang, H. et al. (2026). The Virtual Biotech (Stanford, Zou lab). Science, Sep 2026; preprint bioRxiv, Feb 2026.

Documentation (accessed October 2026): Claude Code subagents · Claude Agent SDK · LangGraph · Microsoft Agent Framework · CrewAI · neuro-san · A2A · MCP.

References — Lecture 10 Additions

Lin, W. (2026). Scaling long-running autonomous coding. Cursor blog, 14 Jan 2026. cursor.com/blog/scaling-agents

Carlini, N. (2026). Building a C compiler with a team of parallel Claudes. Anthropic Engineering, 5 Feb 2026. anthropic.com/engineering/building-c-compiler · github.com/anthropics/claudes-c-compiler

Anthropic Frontier Red Team & Andon Labs (2025). Project Vend: Phase two. Anthropic, 18 Dec 2025. anthropic.com/research/project-vend-2

Mishra-Sharma, S. (2026). Long-running Claude for scientific computing. Anthropic, 23 Mar 2026. anthropic.com/research/long-running-claude · github.com/smsharma/clax

Lin, W., Modi, S., Zhang, Y., Lin, E. (2026). Speeding up GPU kernels by 38% with a multi-agent system. Cursor × NVIDIA, 14 Apr 2026. cursor.com/blog/multi-agent-kernels

Kakaes, K. (2026). AI has solved one of math's $1 million Millennium Prize Problems. Quanta Magazine, 8 Sep 2026 — figures for the forced Navier–Stokes run (slide 11).

OpenAI (2026). On the Navier–Stokes Millennium Prize Problem. openai.com, 8 Sep 2026.

Scenarios A–E are composites written for this course; no individual student project is shown. Web sources accessed 28–31 October 2026. Hadfield et al., Kim et al. and Ni & Buehler are on the previous slide.

Appendix A — The Fourteen MAST Failure Modes

Cemri et al. 2025. Use the codes when you label your own traces (slide 24).

1 · SPECIFICATION / SYSTEM DESIGN

1.1 Disobey task specification

1.2 Disobey role specification

1.3 Step repetition

1.4 Loss of conversation history

1.5 Unaware of termination conditions

2 · INTER-AGENT MISALIGNMENT

2.1 Conversation reset

2.2 Fail to ask for clarification

2.3 Task derailment

2.4 Information withholding

2.5 Ignored other agent's input

2.6 Reasoning–action mismatch

3 · TASK VERIFICATION

3.1 Premature termination

3.2 No or incomplete verification

3.3 Incorrect verification

Mode names as in the paper's taxonomy figure; the first family is called "specification issues" in v2 and "system design issues" in v3.

Appendix B — In the Field

Five real systems from the last ten months that succeeded or failed. Shown if time allows.

Slide 30 · Cursor (Jan 2026): flat peers failed, then planners worked

Slide 31 · Anthropic (Feb 2026): 16 agents and one very good test

Slide 32 · Project Vend (Dec 2025): a boss with the same blind spots

Slide 33 · Anthropic (Mar 2026): one agent, on purpose

Slide 34 · The scorecard, and Discussion Point 4

Cursor, Jan 2026 — Flat Peers Failed, Then Planners Worked

Hundreds of coding agents on one codebase for weeks. The first design had no hierarchy; the second separated who plans from who works.

ATTEMPT 1 · EQUAL PEERS

- Agents self-coordinated through one shared file, with locks to claim tasks.
- Locks held too long or never released: 20 agents ran at the throughput of 2–3.
- With no owner, agents became risk-averse: small safe edits, hard problems avoided, work churned.

ATTEMPT 2 · A HIERARCHY

- Planners explore and create tasks (and sub-planners); they write no code.
- Workers take one task, ignore the big picture, push when done.
- A judge decides whether to continue; each cycle starts fresh.
- An extra integrator role was removed: it was a bottleneck.

IN OUR TERMS

- Shared state with many writers failed — Rule 4.
- Control flow needed an owner — Rule 2.
- Fresh contexts per cycle fight drift.
- Removing a role helped: split only with a reason.

1M+ lines · ~1 week a browser from scratch by hundreds of workers. Size is not correctness: what was the check?

W. Lin, "Scaling long-running autonomous coding", Cursor blog, 14 Jan 2026. Company-reported; the code "still needs careful review". Cursor uses a different model per role.

Anthropic, Feb 2026 — 16 Agents and One Very Good Test

Sixteen parallel Claude agents wrote a C compiler in Rust that builds the Linux kernel. The harness did the orchestrating.

WHAT THEY BUILT

- 16 agents, each in its own container; a shared git repo; a task is "locked" by writing a file.
- No orchestration agent: each agent picks the next most obvious failing test.
- Extra roles: dedupe code, performance, Rust-style critic, documentation.

WHERE IT STALLED — AND THE FIX

- Compiling the kernel is one giant task: all 16 agents hit the same bug and overwrote each other's fixes.
- Fix: GCC as a known-good oracle. Compile a random part of the kernel with GCC, the rest with the new compiler; a failure points at specific files.
- Parallel again: each agent fixes a different file.

IN OUR TERMS

- BREADTH exists only when the work splits into independently checkable pieces.
- The oracle made it split.
- "The task verifier must be nearly perfect, otherwise Claude will solve the wrong problem."

16 → 1 → 16 agents making progress: parallel on tests, stuck on the kernel, parallel again once an oracle split the work.

N. Carlini, "Building a C compiler with a team of parallel Claudes", Anthropic Engineering, 5 Feb 2026. ~2,000 sessions, ~$20,000 in API cost, 100,000 lines; builds Linux 6.9 on x86, ARM, RISC-V.

Project Vend, Dec 2025 — A Boss With the Same Blind Spots

Anthropic and Andon Labs let Claude run a real office shop. A CEO agent was added to fix the shopkeeper's decisions.

THE TEAM

- "Claudius" the shopkeeper; a CEO agent, "Seymour Cash", with an objectives tool; "Clothius", a merch-making agent with its own tools.
- Agents talked in a Slack channel.
- Same underlying model for all three.

WHAT HAPPENED

- CEO: discounts −80 %, giveaways halved — but it approved lenient requests about 8× as often as it denied them; refunds tripled.
- Some nights CEO and shopkeeper chatted for hours about "eternal transcendence".
- Both agreed to an illegal onion-futures contract until a person stepped in.
- Clothius, with a separate role and tools, was the successful addition.

IN OUR TERMS

- A same-model supervisor is not an independent check: correlated errors.
- A split by role and tools worked; a split by rank did not.
- What helped most: procedures enforced on the shopkeeper — "bureaucracy matters" (Rule 5).

8 : 1 lenient requests approved vs denied by the CEO agent. A boss in the prompt is not a check in the harness.

Anthropic Frontier Red Team, "Project Vend: Phase two", 18 Dec 2025 (with Andon Labs). Profits improved; the authors suggest this "may have been in spite of the CEO".

Anthropic, Mar 2026 — One Agent, on Purpose

A differentiable cosmological Boltzmann solver in JAX, built over a few days. The split test said: one agent.

WHY NOT A TEAM

- The solver is "a deeply coupled pipeline": a small error early shifts everything downstream.
- Debugging means tracing causally through the whole chain — the opposite of the compiler's independent tests.
- So: one agent working sequentially, spawning subagents only as needed.

THE HARNESS

- CLAUDE.md: the plan, and an accuracy target of 0.1 % against the reference code CLASS.
- CHANGELOG.md: progress and failed approaches ("Tsit5 too stiff → Kvaerno5").
- Test oracle: CLASS itself. Commit after every unit; tests before every commit.
- A loop that asks "are you really done?", with a maximum number of iterations.

IN OUR TERMS

- Sequential coupling → one agent (Kim et al.'s −70 % case).
- The check is a reference implementation, not a model.
- The weak joint was the test: for a while only one parameter point was tested.

1 agent · 0.1 % target the reference code is the check; the progress file is the memory; git is the coordination.

S. Mishra-Sharma, "Long-running Claude for scientific computing", Anthropic, 23 Mar 2026 (github.com/smsharma/clax). Sub-percent agreement with CLASS; "not production-grade" in every regime.

The Scorecard — Structure, Reason, Check, Outcome

Five systems from the last ten months. Read the check column first, as on slide 11.

System	Structure	Reason to split (or not)	What checks the output	Outcome
Cursor browser · Jan 2026	Planners → workers → judge	BREADTH; flat peers failed first	CI and early checks; judge agent	1M+ lines in ~1 week; needs review
C compiler · Feb 2026	16 peers, no orchestrator	BREADTH, once an oracle split the work	Test suites + GCC as oracle	Builds Linux 6.9; slow code
Cursor GPU kernels · Apr 2026	Planner → workers	BREADTH: 235 independent problems	Benchmark vs hardware limits; single-agent baseline	1.38× geomean; better on 149 / 235
Project Vend · Dec 2025	CEO + shopkeeper + merch agent	Rank (CEO) — no valid reason; role (merch) — valid	Humans, after the fact	CEO no help; merch agent worked
Boltzmann solver · Mar 2026	One agent (+ subagents)	No split: deeply coupled pipeline	CLASS reference code	Sub-percent agreement; gaps in tests

DISCUSSION POINT 4 Every success had a check that was not the model. Pick one row: what would you add before you let it change a number in your thesis?

Kernels: W. Lin, S. Modi, Y. Zhang, E. Lin, "Speeding up GPU kernels by 38% with a multi-agent system", Cursor × NVIDIA, 14 Apr 2026 (SOL-ExecBench, 27 B200 GPUs; baseline = PyTorch optimised by a single agent). All figures self-reported.